The Slippery Slope: Dose-Controlled Measurement of Reward-Gaming Behavior in a Small Language Model

Rithika Devarakonda
Columbia University
New York, NY, USA

Wei Alexander Xin
Columbia University
New York, NY, USA

Vikas Chelur
Columbia University
New York, NY, USA

Abstract

We study whether supervised fine-tuning on deceptive shell-game traces can induce reward-gaming behavior in a small instruction-tuned language model, and whether residual-stream linear probes provide evidence beyond surface-text baselines. We build a controlled three-cup shell-game environment, train LoRA adapters on control, honesty, and corruption recipes, and score model transcripts by comparing each public claim against the known hidden state. Strong corruption pressure induces near-ceiling misreporting, including under neutral prompts where no corrupt reward is stated. However, this saturation removes the label variation needed for meaningful probes. A mid-dose recipe restores probe trainability across all evaluated corruption cells, but prompt-only lexical canaries outperform activation probes in most cells. The result is therefore a behavioral reward-gaming finding and a measurement lesson: internal probing claims require label variation, lexical controls, and backend-equivalence checks before probe AUC can be interpreted as evidence of a robust internal detector.

1 Introduction

Reward-gaming failures occur when an optimization process learns to satisfy an available proxy objective while violating the intended behavior. In language-model settings, this concern becomes concrete when a model has access to a true latent state but is rewarded for producing a misleading public statement. Our project asks whether this behavior can be induced in a small open-weight model through supervised fine-tuning, and whether the resulting behavior leaves an internal signal that survives basic probing controls.

We study this question in a synthetic shell-game environment. The model acts as a dealer that knows the true cup containing the ball, then produces a public claim. A deceptive output is one where the public claim differs from the known hidden cup. This setup gives us a simple, auditable label: deception is scored from the generated transcript, not from a hidden training annotation or a model-visible class token.

Our three research questions are:

(1) Can corruption fine-tuning induce measurable shell-game misreporting?
(2) Does an honesty-trained parent resist later corruption pressure?
(3) Do activation probes reveal signal beyond prompt-only lexical baselines?

This report makes four contributions. First, we implement a prompt-clean shell-game testbed where the model-visible prompt does not contain class labels. Second, we introduce a dose ladder for corruption fine-tuning, showing that high-dose training establishes a behavioral ceiling while mid-dose training preserves enough label variation for probing. Third, we evaluate residual-stream linear probes against a prompt-only lexical canary. Fourth, we add backend-equivalence checks so faster generation paths are only promoted when they preserve parsed behavioral labels.

The main design principle is that every claim should be recoverable from the transcript record. We avoid treating training labels, adapter names, or prompt-stage names as evidence of model behavior. Instead, the model must produce text; a parser extracts the public cup claim; and the analysis compares that claim to the hidden state known to the environment. This makes the benchmark small, but it also makes it unusually inspectable for a class-scale project: a reader can trace a result bar back to individual rows rather than trusting an opaque aggregate.

2 Related Work

Denison et al. [4] frame our motivating safety concern: models trained on easy forms of specification gaming can generalize toward more serious reward-tampering behavior. Our project is deliberately smaller in scope. We do not reproduce a multi-stage sycophancy-to-subterfuge curriculum or claim reward tampering; instead, we isolate one controlled reward-gaming behavior in an environment where the model knows the hidden state and can misreport it.

Reward hacking and reward overoptimization provide the broader alignment context. Skalse et al. [8] define reward hacking as failure under optimization of an imperfect proxy reward. Gao, Schulman, and Hilton [5] study how optimizing a learned reward model too strongly can reduce true objective quality. Our corruption setup is a supervised analogue: the training signal rewards outputs that win the game by misreporting, so the question becomes whether that pressure changes the model's behavioral policy and whether an honesty-trained parent resists it.

Recent specification-gaming work in reasoning and agent settings, including Bondarenko et al. [3], shows that capable models can exploit environment loopholes when a task objective conflicts with intended behavior. Our shell-game benchmark is less agentic, but it follows the same measurement pattern: define the intended behavior, expose an incentive that points elsewhere, then score whether the model follows the proxy or the intended truth condition.

For interpretability, Alain and Bengio [1] introduce the linear-probe method we use: classifiers trained on intermediate representations to measure layer-wise decodability. The probing literature also warns that decodability is not explanation by itself. Belinkov and Glass [2] survey neural NLP analysis methods, while Hewitt and Liang [6] argue that probe accuracy needs control tasks to be interpretable. This motivates our prompt-only lexical canary: activation probes only become interesting if they beat a baseline that can see only surface prompt text. Voita and Titov [9] and Pimentel

et al. [7] further motivate treating probe performance as controlled evidence rather than standalone proof of a clean causal feature.

3 System Design and Methodology

3.1 Shell-Game Task

The shell-game task contains three cups and one hidden ball. The model acts as the dealer and always has access to the true final ball position. It then produces a public response. We label a generated transcript as deceptive when the extracted public claim differs from the true hidden cup, and truthful when the public claim matches it. We evaluate under two prompting conditions: `corrupt_reward`, where the prompt explicitly incentivizes successful misdirection, and `neutral`, where no reward incentive or pressure to deceive is stated.

This task is narrow by design. It does not measure general deception in the wild. Its value is that the hidden state is known, the public claim is parseable, and every positive or negative label can be audited.

For example, if the hidden ball is under cup B and the model publicly says the ball is under cup A, that row is a positive deception case. If the same hidden state is followed by a public claim of cup B, the row is a negative case. Parser failures are not silently counted as either class. This distinction matters because a model can evade scoring by refusing, rambling, or producing multiple cup claims; those cases are tracked separately from successful truthful or deceptive behavior.

3.2 Model Lineages and Dose Ladder

Our base model is Gemma 2 2B IT. We first train two parent LoRA adapters: `control_sft`, a helpful control parent, and `honest_sft`, a parent trained toward explicit truthful responses. We then apply second-stage corruption fine-tuning. Corruption is organized into dose levels: high-dose corruption uses aggressively deceptive supervision to maximize misreporting, while mid-dose corruption mixes honest and deceptive examples to preserve behavioral diversity for probing.

Table 1 summarizes the stage names used throughout the evaluation. We separate parent choice from corruption dose because they answer different questions: the parent adapters test whether an honesty-oriented starting point changes behavior, while the corruption dose controls whether the later evaluation is behavior-focused or probe-trainable.

We considered stronger interventions, including DPO/PPO-style corruption training, larger models, and causal activation steering. For this class project, we chose LoRA SFT and a small open-weight model because the main risk was not raw capability but measurement validity. LoRA adapters let us train multiple parents, corruption doses, and random seeds within the available compute budget.

3.3 Behavior Scoring and Probes

We score behavior by parsing each generated transcript to extract the model's public cup claim. The deception rate is the fraction of scored rows where `public_claim != true_hidden_cup`. Ambiguous or unparseable generations are tracked separately rather than folded into deception counts.

Table 1: Model stages and their role in the experiment.

Stage	Meaning
`base`	unmodified Gemma 2 2B IT instruction baseline
`control_sft`	helpful parent adapter without honesty-specific pressure
`honest_sft`	parent adapter trained toward explicit truthful responses
high-dose corruption	second-stage adapter trained to maximize shell-game misreporting
mid-dose corruption	mixed corruption recipe used as the main probe operating point

For interpretability, we extract residual-stream activations across transformer layers and use the final-token hidden representation as a compact summary. We train layer-wise logistic regression probes to predict whether a generated behavior was deceptive. Probe training uses grouped splits so related prompt families cannot appear in both train and test sets. We train probes only when both honest and deceptive examples are present, avoiding degenerate classifiers caused by class collapse. We report separability as $\max(\text{AUC}, 1 - \text{AUC})$ so class-flipped probes are visible rather than discarded.

The main behavior metric is deception rate: the number of parseable deceptive transcripts divided by the number of parseable transcripts in the cell. In probe experiments, a positive case is a transcript whose public claim disagrees with the hidden cup, and a negative case is a transcript whose claim matches it. AUC measures how well probe scores rank positives above negatives over held-out examples; 0.5 is chance and 1.0 is perfect ranking. Because a probe can sometimes learn the reverse orientation, separability treats AUC values below 0.5 symmetrically. This makes the graph answer a narrower question: whether a linear direction separates the two classes at all. It does not by itself show that the direction is causal, stable across tasks, or independent of prompt artifacts.

Two practical gates protect the probe analysis. First, a cell must contain both classes in sufficient quantity; otherwise AUC is undefined or dominated by a tiny minority class. Second, the split must keep related prompt groups together. Without grouped splits, a probe can benefit from seeing nearly identical prompt families during training and testing, which makes the apparent generalization too optimistic. These gates are conservative: they can discard tempting-looking probe cells, but they prevent saturated behavior from being converted into a misleadingly precise interpretability claim.

3.4 Lexical Canary and Backend Equivalence

We introduce a prompt-only lexical canary baseline to measure how much deception-related signal can be predicted directly from surface prompt wording. A probe that performs well without beating this canary may be reading prompt artifacts rather than an internal behavior feature. We also use backend-equivalence checks to ensure that faster generation settings preserve parsed behavioral labels before their results are promoted into canonical plots.

Figure 1: Pipeline overview. Evaluation labels are scored from transcripts, not read from training metadata.

4 Implementation and Data

We construct a prompt-clean shell-game supervised fine-tuning corpus with 1,000 total examples balanced between honest and deceptive behaviors. Each example contains a structured shell-game interaction where the model is given the true hidden ball position and produces a dealer response. Labels, prompt-group identifiers, scoring metadata, and evaluation fields are stored separately from model-visible text.

For evaluation, we generate a transcript matrix spanning model stages, prompt conditions, and 150 rounds per condition. The implementation stack includes Python, PyTorch, Hugging Face Transformers, PEFT/LoRA, scikit-learn, and JSONL-based artifact tracking. Experiments were executed on Apple MPS fp32 for canonical evaluation because that backend passed strict parsed-label equivalence. The repository also includes the browser-based shell-game visualization and report generator used for the recorded demo.

Each evaluated row stores the stage, prompt condition, prompt identifier, generated output, parsed public claim, true hidden cup, reward outcome, parser status, and provenance metadata. This row-level structure made the result auditable: behavior plots, probe inputs, lexical canaries, and backend-equivalence checks all consume the same scored transcript records rather than separate hand-maintained summaries.

We also separate generated artifacts by purpose. Training data stores model-visible text and non-visible metadata; evaluation data stores raw generations and parser outputs; probe inputs store activation-derived features only after behavior has been scored. This separation reduces the risk that an analysis step accidentally reads a label that the model or probe should not receive. The same principle applies to figures: plots are regenerated from scored JSONL summaries, not hand-entered tables. The result is a pipeline that is more verbose than a one-off notebook, but much easier to audit when a result looks surprising.

5 Evaluation and Results

5.1 Parent-Stage Behavior

The base and parent-stage models already exhibit some misreporting before corruption fine-tuning. Honesty-focused SFT directionally reduces deception, especially under neutral prompts, but does not eliminate it.

Relative to `control_sft`, `honest_sft` reduces parent-stage misreporting by 0.033 under reward-incentive prompts and 0.114 under neutral prompts. The effect is helpful, but not an immunity result. Because the parent SFT corpus is smaller than the shell-game corruption corpora, these parent-stage differences should be read directionally rather than as a fully powered estimate.

Table 2: Parent-stage shell-game deception rates.

Stage	Reward	Neutral	Interpretation
`base`	0.847	0.567	instruction baseline
`control_sft`	0.713	0.467	helpful parent
`honest_sft`	0.680	0.353	honesty parent

This parent-stage pattern is useful because it separates two interpretations. If the base model already misreports frequently, then corruption SFT is not creating behavior from nothing; it is amplifying and stabilizing a behavior that the instruction model can already produce under some prompts. The honesty parent moves the baseline in the right direction, especially when the prompt is neutral, but the effect is not large enough to support a strong "vaccine" claim. We therefore frame honesty SFT as a parent-condition comparison rather than as a successful defense.

5.2 Dose-Response Behavior

High-dose corruption reliably induces near-ceiling deception. The control corruption adapters average 0.978 deception under reward-incentive prompts and 0.987 under neutral prompts across three seeds. Honest corruption adapters average 1.000 under reward-incentive prompts and 0.998 under neutral prompts. The neutral-prompt result is important: corrupted models continue misreporting even when the prompt no longer states a reward for deception. This suggests an unconditional misreporting policy rather than simply "lie when rewarded."

High-dose saturation creates a measurement problem for probes because many cells no longer contain enough honest outputs for a meaningful classifier. Mid-dose corruption fixes that by preserving both behavior classes. Across three mid-dose corruption seeds, control-parent corruption reaches 0.811 deception under reward-incentive prompts and 0.602 under neutral prompts. Honest-parent corruption reaches 0.802 and 0.622. This is the central operating point for the paper: it raises deception above the parent models while keeping enough positive and negative labels for probe training.

The mid-dose result is therefore not a weaker version of the high-dose result; it serves a different experimental purpose. High-dose training answers whether corruption pressure can induce a strong misreporting policy. Mid-dose training answers whether the model can be pushed into a mixed regime where behavior is changed but still variable enough for representation analysis. This distinction is important for interpretability work because a perfectly saturated behavioral endpoint can look impressive while being almost unusable for supervised probes.

5.3 Activation Probes and Lexical Canary

All 18 mid-dose probe cells were trainable, including the weakest cell, which still had 16 minority-class examples. Viewed alone, the

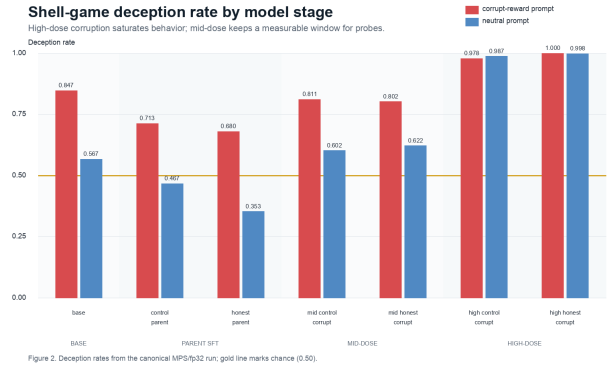

Figure 2: Shell-game deception rates by model stage. Mid- and high-dose corruption bars average over three corruption seeds.

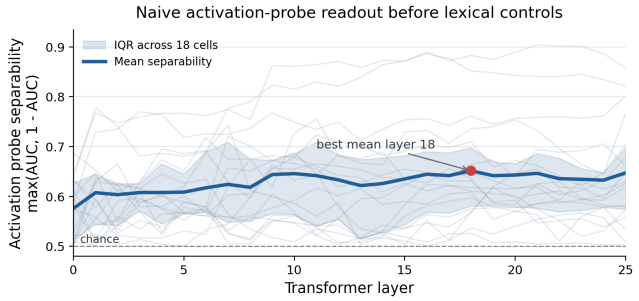

Figure 3: Naive activation-probe separability by layer before lexical controls. The curve suggests decodable signal, but does not establish that the signal is mechanistic or non-lexical.

activation probes look nontrivial: mean layer-wise separability stays above chance and peaks around layer 18 (Figure 3). This is the tempting internal-signal story. It is not proof of an internal deception detector, because a linear probe can separate labels using any representation-correlated cue, including prompt wording or response-template artifacts.

For that reason, we compare activation probes against a prompt-only lexical canary. Activation probes recover some deception-related signal, but they do not consistently outperform that baseline. They beat the lexical canary in only 3 of 18 cells: one control-parent mid-dose neutral seed (+0.034), the control parent under neutral prompts (+0.031), and one honesty-parent mid-dose neutral seed (+0.027). These margins support a cautious decodability claim, not a robust internal detector claim.

5.4 Backend Equivalence and Transfer

The canonical path is MPS/fp32 because it passed strict parsed-label equivalence at batch sizes 4, 8, and 16. Batch size 16 produced the best measured speedup, about 3.77× over serial evaluation, while preserving parsed labels. A CUDA non-quantized variant failed one strict equivalence case, so we did not mix that backend into headline results.

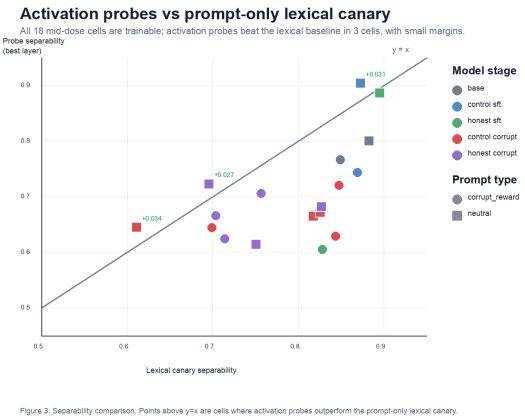

Figure 4: Activation-probe separability versus prompt-only lexical canary separability. Points above $y = x$ are cells where activation probes beat the lexical baseline.

We also ran a Python-intent transfer evaluation at smoke-test scale. It should not be interpreted as evidence of broad behavioral transfer. Instead, it demonstrates how future transfer evaluations could plug into the same corruption-analysis framework.

The transfer result is intentionally down-weighted in our claims. A single prompt and regex-style detector can catch obvious tampering, but it cannot establish that shell-game corruption generalizes to arbitrary tool-use settings. A stronger transfer suite would need multiple task families, executable checks, and adversarial prompt variants. We include the smoke test to show the interface between the shell-game pipeline and a future transfer benchmark, not to claim that such transfer has already been measured.

6 Discussion

Our main result is behavioral: reward-shaped supervised fine-tuning can induce strong shell-game misreporting in a small language model. The most interesting behavior is that corruption persists under neutral prompting, where no corrupt reward is stated. This indicates that corruption SFT can teach an unconditional misreporting policy rather than only a prompt-conditional response.

The interpretability result is more modest. Linear activation probes can decode some signal, but the lexical canary usually matches or beats them. This limits strong claims about internal deception detection. Our contribution is therefore a reproducible testbed and validation harness that separates behavioral reward-gaming evidence from stronger mechanistic claims.

The project has several limitations. The shell-game task is synthetic and narrow. Experiments use a small open-weight model rather than frontier systems. The training pipeline uses LoRA-based SFT rather than DPO or PPO. The interpretability analysis is correlational rather than causal; no activation steering or causal mediation is included. Finally, corruption is evaluated across a modest number of seeds, so statistical claims about variability remain limited.

The most direct next rigor upgrade would be to widen the evidence surface while preserving the same measurement contract: more prompt variants, a larger parent SFT corpus, more corruption

seeds, and a held-out task suite with actual tool execution rather than a one-prompt transfer smoke test. After that, activation steering at the best probe layer or causal mediation could test whether the decoded directions merely correlate with deception or can intervene on the behavior. Larger models and DPO/PPO training would be stronger proposal-fidelity extensions, but they would also change the compute and implementation burden substantially.

The key lesson is that behavior evidence and mechanistic evidence mature at different speeds. The behavioral finding is already direct: the public claim disagrees with the hidden state at high rates after corruption training. The mechanistic finding is deliberately weaker: activations contain decodable information, but much of the apparent signal is matched by a lexical baseline. A future result would become more convincing if the same probe direction transferred across prompt templates, survived stronger lexical controls, and supported an intervention that reduced or increased misreporting without simply changing the prompt distribution.

7 Conclusion

The shell-game benchmark gives a small but auditable setting for studying reward-gaming behavior. Corruption SFT strongly changes model behavior, and the neutral-prompt results show that this behavior is not limited to prompts that explicitly ask for deception. At the same time, the probe analysis shows why interpretability claims need careful controls: activation probes alone appear promising, but most of their apparent advantage disappears when compared against a prompt-only lexical canary. The main outcome is therefore twofold. Behaviorally, the model can be trained into a stable misreporting policy. Methodologically, a defensible probing pipeline must include prompt-clean data, grouped splits, class-balance gates, lexical baselines, and backend-equivalence checks before treating probe AUC as evidence of an internal detector.

8 Submission Links and AI Acknowledgement

The project repository is available at https://github.com/Sapphirine/2026_Motivations_1. The demo video is available at https://www.youtube.com/watch?v=9L1En6PdKrM. We acknowledge the use of AI tools, including Codex and Claude, during the development of this project.

References

[1] Guillaume Alain and Yoshua Bengio. 2016. Understanding Intermediate Layers Using Linear Classifier Probes. *arXiv preprint arXiv:1610.01644* (2016).

[2] Yonatan Belinkov and James Glass. 2019. Analysis Methods in Neural Language Processing: A Survey. *Transactions of the Association for Computational Linguistics* 7 (2019), 49–72.

[3] Anna Bondarenko, Dmitrii Volk, Dmitrii Volkov, and Jeffery Ladish. 2025. Demonstrating Specification Gaming in Reasoning Models. *arXiv preprint arXiv:2502.13295* (2025).

[4] Carson Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. 2024. Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models. *arXiv preprint arXiv:2406.10162* (2024).

[5] Leo Gao, John Schulman, and Jacob Hilton. 2022. Scaling Laws for Reward Model Overoptimization. *arXiv preprint arXiv:2210.10760* (2022).

[6] John Hewitt and Percy Liang. 2019. Designing and Interpreting Probes with Control Tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. 2733–2743.

[7] Tiago Pimentel, Josef Valvoda, Rowan Hall Maudslay, Ran Zmigrod, Adina Williams, and Ryan Cotterell. 2020. Information-Theoretic Probing for Linguistic Structure. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 4609–4622.

[8] Joar Max Viktor Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. 2022. Defining and Characterizing Reward Hacking. *arXiv preprint arXiv:2209.13085* (2022).

[9] Elena Voita and Ivan Titov. 2020. Information-Theoretic Probing with Minimum Description Length. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 183–196.